

Roteiro da apresentação

Code Burguer's · Qualidade e Teste de Software · Entrega 1 · 12 slides, cerca de 17 minutos

Slides	Quem fala	Parte	Tempo
1	Yuri Mascarenhas	Abertura	1 min
2 e 3	Yuri Mascarenhas	Autenticação, sessões e controle de acesso	3 min
4 e 5	Vinicius Rocha Pinheiro	Carrinho, pedidos e checkout	3 min
6 e 7	Thiago Ferreira	Gestão de clientes e endereços	3 min
8 e 9	Vinicius Fonseca de Freitas	Cardápio e montagem de lanches	3 min
10 e 11	Felipe Paixão	Gestão de estoque e insumos	3 min
12	Felipe Paixão	Encerramento	1 min

Como usar: cada pessoa tem a sua página. O texto está quebrado em uma ideia por linha, para ler olhando pra turma e voltando o olho rápido. Onde está escrito PAUSA, respira e troca o slide — é o momento em que a turma processa o que você acabou de falar. Não precisa decorar nem ler palavra por palavra: o que não pode é você não saber explicar o teste que está citando.

Yuri Mascarenhas

Abertura · Slide 1 · 1 min

SLIDE 1

Boa noite, pessoal.

A gente é o grupo do Code Burger's.

É um sistema de lanchonete online, feito em Java, com Servlet e JDBC.

— PAUSA —

Só que não fomos nós que escrevemos esse sistema.

Ele veio pronto, de um projeto antigo.

O nosso trabalho foi outro: testar ele.

E achar o que está quebrado.

— PAUSA —

A gente dividiu em cinco áreas, uma pra cada um.

Cada pessoa tem dois slides:

o que testou, e o que encontrou.

Adianto que encontramos bastante coisa.

Yuri Mascarenhas

Autenticação, sessões e controle de acesso · Slides 2 e 3 · 3 min

SLIDE 2

Minha parte é login e sessão.

A classe chama ValidadorCookie.

— PAUSA —

O que ela faz dá pra explicar em uma frase:
chega uma requisição, ela olha os cookies,
procura o cookie chamado token,
e pergunta pro banco se aquele token existe.

— PAUSA —

Por que ela não é CRUD?

Porque ela não salva e não lê tabela nenhuma.

Ela decide. Ela olha o nome do cookie e escolhe o caminho.

— PAUSA —

Nos testes o banco é falso, a gente usa Mockito.

Então roda em menos de um segundo, sem subir nada.

— PAUSA —

E não testamos só o caminho feliz.
Testamos requisição sem cookie nenhum,
token de cliente tentando entrar como funcionário,
e esse aí do slide: cookie repetido.

SLIDE 3

26 testes passando.
E os 22 ramos da classe cobertos.
Ou seja: todo if e todo else foi executado pelo menos uma vez.

— PAUSA —

Agora a parte interessante.
A gente foi procurar onde o token expira.

— PAUSA —

E descobriu que ele não expira.

— PAUSA —

No servidor, a tabela guarda só o texto do token. Nada mais.
Quem expira é o cookie no navegador, em 30 minutos.
Só que se alguém copiar aquele token, ele funciona pra sempre.

— PAUSA —

A gente não inventou uma regra que não existe.
Registrou como observação de segurança, e está no slide.

— PAUSA —

E o teste manual já rodou: oito passos, todos passaram.
Login, logout e bloqueio de página restrita funcionam.

Para abrir na hora: ValidadorCookieLimitesTest · mvn test -Dtest=ValidadorCookie*

Vinicius Rocha Pinheiro

Carrinho, pedidos e checkout · Slides 4 e 5 · 3 min

SLIDE 4

Minha parte é o carrinho e o fechamento do pedido.
A classe é o servlet comprar.

— PAUSA —

Ele tem regra de verdade:
valida o cookie, lê o JSON que o carrinho manda,
separa o que é lanche e o que é bebida,
soma o total, e grava o pedido com os itens.

— PAUSA —

O detalhe importante do meu teste é esse:
eu não acredito na mensagem que o sistema responde.

— PAUSA —

Eu pego o Pedido que ele manda pro banco e abro pra conferir.
O total, o cliente, a quantidade de cada item.

— PAUSA —

Porque dá pra um teste passar só olhando

a mensagem Pedido salvo com sucesso,
e o pedido estar todo errado por dentro.

SLIDE 5

22 testes passando.

E dois vermelhos de propósito. Já explico o porquê.

— PAUSA —

Põe no carrinho dois lanches de quinze reais
e um refrigerante de cinco.

A tela mostra trinta e cinco.

— PAUSA —

O que o sistema grava no banco? Vinte.

— PAUSA —

Ele soma o preço uma vez por item e ignora a quantidade.
Na prática: quanto mais você compra, mais barato fica.
Essa é a issue vinte e um.

— PAUSA —

E tem outra, a vinte e dois, que é pior.

O id do cliente vem do JSON que o navegador manda.

O sistema nunca compara com o cookie de quem está logado.

— PAUSA —

Então eu, logado na minha conta,
consigo fazer um pedido no nome de qualquer um de vocês.

— PAUSA —

E o teste manual achou mais duas.

Finalizar a compra com bebida no carrinho devolve erro 500.

E carrinho vazio grava um pedido de zero real.

Para abrir na hora: `mvn test -Pdefeitos -Dtest=ComprarRegrasTest · mostrar 'expected: <35.0> but was: <20.0>'`

Thiago Ferreira

Gestão de clientes e endereços · Slides 6 e 7 · 3 min

SLIDE 6

Minha parte é o cadastro e o login do cliente.

São três classes: o DaoCliente, o servlet de cadastro e o EncryptadorMD5.

— PAUSA —

O DaoCliente é um DAO, então tem bastante CRUD nele.

Mas o método login tem uma decisão de verdade:

só entra se o hash da senha bater

e se o cliente estiver ativo.

— PAUSA —

Uma coisa que a gente mudou e que vale comentar:

o teste que já existia usava Objenesis e reflexão

pra conseguir enfiar um banco falso dentro da classe.

Funcionava, mas ninguém entendia ao ler.

— PAUSA —

A gente trocou por um construtor normal, que recebe a conexão.

Ficou bem mais simples.

— PAUSA —

E o hash esperado a gente não tirou do próprio sistema.
Se o sistema estiver errado, o teste erraria junto.
Pegamos da tabela oficial do MD5.

SLIDE 7

29 testes passando.
E aqui aparecem dois problemas.

— PAUSA —

O primeiro: o cadastro aceita campo vazio.
Nome vazio, usuário vazio, senha vazia.
O servidor grava e responde Usuário cadastrado.

— PAUSA —

Aí vocês perguntam: mas a tela não valida?
Não valida.

— PAUSA —

Tem um if no JavaScript escrito errado,
que nunca é verdadeiro.

Então aquele alerta de campo obrigatório nunca aparece.

Essa é a issue vinte e quatro.

— PAUSA —

O segundo é a busca por usuário.

Ela monta o SQL colando texto.

— PAUSA —

Então se a pessoa se chama O'Brien, com apóstrofo, o login quebra.

E se ela for esperta, digita SQL no campo de usuário

e o banco executa.

Issue vinte e três.

— PAUSA —

No teste manual apareceram mais duas.

Dá pra cadastrar duas pessoas com o mesmo usuário,

e quem entra no login é a última.

E nome com acento grava corrompido no banco.

Para abrir na hora: DaoClienteTest (deveRecusarClienteInativo) · cadastro.js, a condição `!field.name == 'complemento'`

Vinicius Fonseca de Freitas

Cardápio e montagem de lanches · Slides 8 e 9 · 3 min

SLIDE 8

Minha parte é a montagem do lanche personalizado.
Aquele tela onde o cliente escolhe pão, carne, queijo, bacon.

— PAUSA —

O DaoLanche é CRUD puro,
então a regra que eu testei está no servlet.

— PAUSA —

Ele faz três coisas:
calcula o preço somando ingrediente por ingrediente,
salva o lanche como inativo, porque não é do cardápio,
e liga cada ingrediente ao lanche.

— PAUSA —

No teste eu faço a conta na mão:
dois queijos, três bacons e um pão dá dezoito e cinquenta.

— PAUSA —

Se eu deixasse o teste usar a conta do próprio código,

o código poderia mudar e o teste mudaria junto, sem reclamar.
Fazendo na mão, ele reclama.

SLIDE 9

Esse slide é o meu favorito.

— PAUSA —

A gente já desconfiava que os ingredientes não estavam sendo salvos.
E a suspeita era do DaoLanche.

— PAUSA —

Fomos olhar: o DaoLanche está certo.
Ele executa o INSERT normalmente.

— PAUSA —

O problema está no servlet.
Tem dois laços lá, um embaixo do outro, usando o mesmo Iterator.

— PAUSA —

O primeiro laço, o do preço, consome tudo.
Quando chega no segundo, o que liga os ingredientes,
não sobrou nada. Ele simplesmente não roda.

— PAUSA —

O teste esperava três ingredientes ligados ao lanche.
Recebeu zero.

— PAUSA —

E olha que interessante: isso bate com o teste manual.
Dia vinte eu montei um lanche na mão,
cliquei em Adicionar, e não aconteceu nada.
Virou a issue dezessete.
Agora a gente sabe o motivo, que é a issue vinte.

Para abrir na hora: `salvarLancheCliente.java`, os dois `while(keys.hasNext())` · `mvn test -Pdefeitos -Dtest=SalvarLancheClienteTest`

Felipe Paixão

Gestão de estoque e insumos · Slides 10 e 11 · 3 min

SLIDE 10

Minha parte é estoque e insumos.

E eu vou ser honesto com vocês sobre uma coisa.

— PAUSA —

O requisito pedia uma classe que não fosse CRUD, com uma complexidade razoável.

Na minha área, essa classe não existe.

— PAUSA —

O DaoIngrediente é CRUD do começo ao fim.

E o sistema não tem regra de estoque nenhuma: quando alguém compra, o estoque não baixa.

— PAUSA —

Então a gente fez duas coisas.

Cobriu o DAO, que era o que a issue cinco pedia.

E pegou como classe não CRUD os servlets do painel, que pelo menos decidem quem pode entrar e quem não pode.

— PAUSA —

Essa é a limitação do nosso trabalho, e ela está escrita no slide.
A gente preferiu deixar visível
a inventar uma classe só pra fechar cinco nomes.

SLIDE 11

16 testes passando.

— PAUSA —

O defeito daqui: o painel aceita número negativo.

— PAUSA —

Cadastra um insumo com quantidade menos cinco, ele aceita.
Preço de compra negativo, aceita também.
E responde Ingrediente salvo.

— PAUSA —

Não tem validação no servlet,
e o banco também não tem restrição nenhuma.
Issue vinte e cinco.

— PAUSA —

E tem uma coisa que a gente decidiu não fazer.

— PAUSA —

Não escrevemos teste dizendo
que preço de venda tem que ser maior que o de compra.
Parece óbvio, mas não existe nada no sistema que diga isso.
A gente testa o que está escrito, não o que a gente acha.

— PAUSA —

No manual apareceu mais uma:
remover um insumo que está sendo usado num lanche
devolve um erro 500 na cara do usuário.

Para abrir na hora: `IngredienteControllersTest (deveBloquearQuemNaoEhFuncionario e caracterizacaoValoresNegativosSaoAceitos)`

Felipe Paixão

Encerramento · Slide 12 · 1 min

SLIDE 12

Fechando.

— PAUSA —

115 testes rodando, todos passando.

— PAUSA —

Mais cinco testes que ficam vermelhos de propósito.

Eles descrevem como deveria funcionar,
e falham enquanto o defeito existir.

— PAUSA —

E onze defeitos no total: seis já viraram issue, da vinte até a vinte e cinco.

— PAUSA —

O que ficou faltando, e a gente não vai esconder:
cinco achados dos testes manuais ainda não viraram issue.
E a área de estoque ficou sem classe de regra de negócio,
como eu mostrei ali atrás.
E a gente não corrigiu nenhum defeito,

porque o objetivo dessa entrega era encontrar e registrar.

— PAUSA —

Uma última coisa, porque é fácil confundir:
cobertura alta não quer dizer que não tem defeito.
Quer dizer só que aquele trecho foi executado.

— PAUSA —

Tanto é que a gente tem cobertura alta e seis defeitos.

— PAUSA —

É isso. Obrigado, e a gente está aberto pra perguntas.

Para abrir na hora: mvn clean test · mostrar 'Tests run: 115, Failures: 0'

Perguntas prováveis

Qualquer um do grupo pode responder

Por que tem teste vermelho de propósito?

Porque ele descreve como deveria funcionar. Enquanto o defeito existir, ele falha. A gente deixou num perfil separado pra não quebrar o build normal, e não usou @Disabled, que só esconderia o problema.

Vocês corrigiram os defeitos?

Não. O objetivo dessa entrega era encontrar e registrar. Cada defeito tem um teste que reproduz ele e uma issue no GitHub.

Por que vocês mexeram no código de produção?

Só adicionamos construtores que recebem a conexão ou os DAOs, pra conseguir usar mocks. O construtor padrão continua igual e o comportamento no Tomcat não mudou.

A classe de estoque não é CRUD mesmo?

É CRUD, e a gente assume isso. O sistema não tem regra de estoque: quando alguém compra, nada baixa. Usamos os servlets do painel como melhor alternativa e deixamos a lacuna registrada.

Cobertura alta quer dizer que está sem bug?

Não. Quer dizer só que aquelas linhas foram executadas pelos testes. Tanto é que a nossa cobertura é alta e mesmo assim achamos seis defeitos.